PHP OOP

مختصر يُعرّف ما هو البرمجة كائنية التوجه، لماذا نستخدمها، وأهم المفاهيم الأساسية التي يجب معرفتها للبدء.

الأساسيات

Class

- الكلاس هو المخطط الذي يُبنى منه الكائن
- Object لا يمكن كتابة أي خاصية أو دالة إلا داخل
- "class User" مثال:

Object

- Class هو نسخة حقيقية من
- "new" كلمة مفتاحية
- Objects يعني إنشاء عدد من Class يمكن أن يُرجع من
- "$u = new User;" مثال:

Property

- Class هو متغير داخلي
- Object يمثل حالة كل
- "public string $name;" مثال:

Method

- Object هي دالة داخلية تمثل سلوك
- "function getInfo() { ... }" مثال:

$this

- Object هو كلمة تشير إلى الـ
- Methods و Properties يستخدم للوصول إلى
- "$this->name;" مثال:

Constructor

- new" يُنفذ تلقائيًا عند "__construct" هي الدالة الخاصة
- Properties هو المكان المناسب لتهيئة
- "function __construct($name) { $this->name = $name; }" مثال:

Constructor Promotion

- PHP 8 اختصار جاء في
- Property يعني تعريف الخاصية وتهيئتها بسطر واحد
- "function __construct(private string $name)" مثال:

Encapsulation والتغليف

Visibility

- "public" هذا يعني أن الشيء متاح من أي مكان
- "private" هذا يعني أن الشيء متاح داخل Class فقط
- "protected" هذا يعني أن الشيء متاح داخل Class والأصناف التي ترث منه

Encapsulation

- Properties هو إخفاء البيانات وتغليفها بداخل الكائن
- "private" الكلمة المفتاحية التي تمنع الوصول إلى
- Methods هي الطريقة الوحيدة المسموح بها للتعامل مع تلك
- **يجب تطبيق هذا المبدأ دائمًا تقريبًا في البرمجة الحديثة**

Getter and Setter

- Getter هو دالة خاصة بقراءة Property هي الدالة المسؤولة عن
- Setter هو دالة خاصة بتعديل Property هي الدالة المسؤولة عن
- "getBalance()" مثال:
- "setAge()" مثال:

Validation

- Setters و Getters ليست مجرد دوال عادية، بل هي المكان المثالي لوضع
- التحقق من القيمة قبل حفظها
- رفض القيمة غير المنطقية بدلًا من تخزينها
- Exception هي الطريقة المعتمدة في PHP لإبلاغ الكود عن حدوث خطأ، بدلًا من
- استخدام

Exceptions

- "throw" هي الكلمة المفتاحية التي
- throw new \InvalidArgumentException("msg"); مثال:
- يُستخدم لإيقاف التنفيذ في نقطة معينة إذا لم يُمسك هذا الخطأ
- "try" يعني أن الكود الذي بداخله قد يُسبب خطأ
- "catch" يعني كيف نتعامل مع الخطأ إذا حدث
- "finally" هو الجزء الذي يُنفَّذ دائمًا، سواء حدث خطأ أم لا

Custom Exception

- Class هي أيضًا مجرد Exception ما هي إلا كلاس يرث من
- "InsufficientFundsException" يمكننا إنشاء أنواع خاصة بنا من الأخطاء، مثل

Static والثوابت

'static'

- Class هذا يعني أن العضو ينتمي إلى Object وليس إلى
- "$logger->log("msg");" مثال:

'self::'

- Class هي طريقة للوصول إلى العضو من داخل نفس
- "self::$count;" مثال:

'const'

- "const PI = 3.14;" مثال:
- "Circle::PI" يُستخدم كالتالي:
- Property هي قيمة ثابتة لا يمكن تغييرها بعد تعريفها

'readonly'

- PHP 8.1 هذه الكلمة جاءت مع إصدار
- Error سيحدث خطأ عند محاولة تعديل خاصية readonly بعد تهيئتها
- "$this->id = 5;" مثال:

Immutable Object

- Object هي الفكرة التي تجعل الكائن لا يتغير بعد إنشائه
- "readonly" يمكنه أن يفعل هذا لكل الخصائص في Objects بضغطة واحدة

الوراثة

Inheritance

- Class هي آلية تسمح لكلاس بأخذ خصائص وسلوكيات كلاس آخر
- Parent Class هو الكلاس الذي نرث منه، ويُسمى أيضًا الأب
- Child Class هو الكلاس الذي يرث، ويُسمى أيضًا الابن

'extends'

- "class Employee extends Person {" مثال:
- الابن يرث الأعضاء المحمية من الأب

'parent::'

- Class هي الطريقة التي يصل بها الابن إلى شيء تابع للأب
- "parent::__construct($name, $age);" مثال:
- يُستخدم لتجنب تكرار الكود، فبدلًا من إعادة كتابة منطق موجود بالفعل في الأب، يمكننا استدعاؤه

Overriding

- Method هي إعادة تعريف دالة ورثناها من الأب
- يمكن الابن مع ذلك تغيير سلوكها بالكامل
- "parent::method()" يمكننا استدعاء نسخة الأب من الدالة بـ

'final'

- "final class" هي الكلمة التي تمنع الوراثة تمامًا:
- "final function" تمنع إعادة تعريف دالة معينة فقط:

'instanceof'

- Object هو عامل يتحقق إن كان كائن ما ينتمي إلى
- "$x instanceof A::class" مثال:

قواعد الوراثة

- لا يمكن للابن الوصول مباشرة إلى الخصائص والدوال من نوع Private في الأب
- يمكن للابن الوصول إلى الخصائص والدوال من نوع Protected
- يجب التمسك بمبدأ "العلاقة هي" عند التفكير في استخدام الوراثة، فلا نستخدمها إلا إذا كان

التجريد والعقود والأشكال

Abstraction

- Abstraction هو مبدأ يركز على إظهار ما يفعله الكائن، وإخفاء التفاصيل المعقدة عن طريقة العمل
- Abstract Class هي كلاس لا يمكن إنشاء Object منه مباشرة، وإنما يُستخدم فقط كقالب يرث منه كلاسات أخرى
- Abstract Methods هي دوال داخل Abstract Class لا تحتوي على تنفيذ، وإنما تكتفي بتعريف الشكل فقط
- Abstract Methods يجب أن تُنفَّذ من قِبل أي كلاس يرث من Abstract Class
- Abstract Class يمكن أن تحتوي على Properties و Methods عادية بالإضافة إلى Abstract Methods
- Abstract Class لا يمكن إنشاء Object منها، لكن يمكن أن يرث منها كلاسات أخرى
- Abstract Class يمكن أن ترث من Abstract Class أخرى، وتستطيع تنفيذ بعض أو كل الدوال المجردة الموروثة

Interface

- Interface هي آلية تسمح لكلاس بأن يلتزم بمجموعة من الدوال دون أن يرث منها
- Interface لا يمكن إنشاء Object منها مباشرة
- Interface لا يمكن أن تحتوي على Properties عادية
- Interface لا يمكن أن تحتوي على Methods مكتوبة بالكامل
- Interface جميع الدوال داخل Interface هي Abstract Methods ضمنيًا، حتى لو لم نكتب
- "implements" هي الكلمة التي نستخدمها لكي يلتزم Class بعقد Interface
- Class يمكن أن يُنفِّذ أكثر من Interface في الوقت نفسه، وهذا أحد أهم أسباب وجودها
- "Flyable" و "Walkable" هذا يُسمى Interface Segregation، وهو أن نُقسّم العقود الكبيرة إلى عقود صغيرة ومحددة
- Interface يمكن أن ترث من Interface أخرى باستخدام "extends"

Interface و Abstract Class الفرق بين

- Abstract Class يمكن أن تحتوي على حالة داخلية عبر Properties، بينما Interface لا تستطيع ذلك
- Abstract Class يمكن أن ترث من Abstract Class واحدة فقط، بينما Class يمكنها أن تُنفِّذ عدد من Interfaces
- Interface يُستخدم عندما نريد تعريف "ما يستطيع فعله" الكائن، بغض النظر عن نوعه الحقيقي
- Abstract Class يُستخدم عندما نريد مشاركة "ما هو" الكائن وبعض السلوك المشترك بين أنواع قريبة

Polymorphism

- Polymorphism هو القدرة على التعامل مع كائنات مختلفة من خلال واجهة موحدة، دون الحاجة إلى معرفة النوع الدقيق لكل منها
- "function pay($payment, $amount)" مثال: دالة واحدة تستقبل أي Object يحتوي على Method اسمها pay، وتستدعيها دون أن تهتم بنوعه
- "Cash" و "Card" و "Crypto" هي ثلاث Classes مختلفة تمامًا، لكن كلًا منها يُنفِّذ Interface واحدة اسمها "Payable"
- "$payment->pay($amount);" هذا السطر يعمل بشكل صحيح مع أي Object يُنفِّذ Payable، دون أن نعرف أو نهتم بنوعه الحقيقي
- Polymorphism يجعل الكود أكثر مرونة، لأنه يعتمد على العقود (Interfaces) بدلًا من التفاصيل (Classes)، ويسهل إضافة أنواع جديدة دون تعديل الكود الموجود

Type Hinting

- Type Hinting هو أن نُحدد نوع البيانات المتوقعة في المعاملات (Parameters) وقيم الإرجاع (Return Values) للدوال
- "function pay(Payable $m)" مثال:
- Payable هنا تعني أن $m يمكن أن يكون أي Object يُنفِّذ Interface اسمها
- Type Hinting يجعل الكود أكثر أمانًا، لأنه يكتشف الأخطاء مبكرًا قبل أن تتسبب في مشاكل

Traits

- Traits هي آلية تسمح بمشاركة مجموعة من الدوال والخصائص بين Classes مختلفة، دون الحاجة إلى وراثة أو تنفيذ Interface
- "use Timestamps;" مثال: داخل Class نكتب
- Timestamps هو اسم Trait نريد استخدامها
- Trait هي أشبه بـ "نسخ ولصق" ذكي للكود، حيث يتم نسخ محتوى الـ Trait إلى داخل الـ Class عند تعريفها

Composition

- Composition هي طريقة لبناء الكائنات المعقدة عن طريق تجميعها من كائنات أبسط، بدلًا من الاعتماد على الوراثة
- Class هي مثال على Composition
- Product و Cart هما Classes مستقلتان، لكن Cart تحتوي على Product داخلها
- Cart هي المسؤولة عن إدارة المنتجات، لكنها لا تعرف كيف يُحسب السعر الإجمالي
- Product هي المسؤولة عن حساب سعرها، لكنها لا تعرف شيئًا عن السلة
- "$cart->total();" مثال: عندما نستدعي هذه الدالة، تطلب Cart من كل Product في السلة سعرها، ثم تجمع النتائج
- Composition أكثر مرونة من الوراثة، لأنها تسمح بتغيير سلوك الكائن أثناء التشغيل، عن طريق استبدال الكائنات المُضمّنة بأخرى

Dependency Injection

- Dependency Injection هي طريقة لتمرير الكائنات التي يحتاجها Class من الخارج، بدلًا من أن تُنشئها بنفسها
- "new" بدلًا من أن نكتب داخل الكلاس، نُمرّر الكائن كمعامل في الدالة البانية أو في دالة أخرى
- implements بدلًا من الاعتماد على Class محددة، نعتمد على Interface تصف ما نحتاجه فقط
- "new UserService(new SmtpMailer);" مثال:
- Method هي أسلوب يسمح باستدعاء عدة Methods متتالية على نفس الكائن، عن طريق إرجاع $this من كل دالة
- "$db->table('x')->where('b')->limit(5);" مثال:

Method Chaining

سمات وخصائص الكلاسات

Magic Methods

- Magic Methods هي دوال خاصة تبدأ بشرطتين سفليتين "__"، وتُستدعى تلقائيًا في مواقف محددة
- PHP لكل منها اسم محجوز لا يمكن استخدامه لشيء آخر
- "__construct" مثال: تُستدعى عند إنشاء Object
- "__toString" مثال: تُستدعى عند طباعة Object
- "__get" و "__set" مثال: تُستدعى عند محاولة قراءة أو كتابة Property غير موجودة أو غير متاحة
- "__call" مثال: تُستدعى عند استدعاء Method غير موجودة أو غير متاحة
- "__isset" مثال: تُستدعى عند استخدام "isset($obj->x)" على Property غير متاحة
- "__unset" مثال: تُستدعى عند استخدام "unset($obj->x)" على Property غير متاحة
- Countable هي واجهة تسمح لكائنك بالعمل مع "count($obj)" مثال: تدعم الدالة العامة
- ArrayAccess هي واجهة تسمح لكائنك بالعمل مع "$obj["key"]" مثال: تدعم الوصول بالأقواس المربعة
- IteratorAggregate هي واجهة تسمح لكائنك بالعمل مع "foreach" مثال: تدعم التكرار
- JsonSerializable هي واجهة تتحكم في كيفية تحويل الكائن إلى JSON مثال: تُستدعى عند
- "json_encode($obj)" استدعاء

Enums

- Enums هي طريقة لتعريف مجموعة محدودة من القيم الثابتة المسموح بها
- PHP 8.1 هذه الميزة جاءت مع إصدار
- "Shipped" و "Paid" و "Pending" مثال: حالات الطلب
- "OrderStatus::Paid" مثال: الوصول إلى إحدى القيم

أنماط التصميم Design Patterns

Singleton

- Singleton هو نمط تصميم يضمن وجود Object واحدة فقط من الكلاس، ويوفر نقطة وصول عامة لها
- "getInstance()" مثال: يُستخدم Constructor خاص و
- Class هو نمط تصميم يُستخدم لإنشاء Objects من
- "NotificationFactory::create("sms");" مثال: تُرجع الدالة Object مختلفة بناءً على المعامل الذي تستقبله

Factory

- Interface هو نمط تصميم يُعرّف عقدًا مشتركًا لمجموعة من الكلاسات، بحيث يمكن استبدال أي منها بالأخرى دون تغيير باقي الكود
- Fixed Discount و Percent Discount مثال: كلاسات مختلفة تُطبّق كل منها طريقة حساب الخصم بشكل مختلف

Strategy

- Subject هو الكائن الذي يحتفظ بقائمة من المراقبين، ويُخطرهم عند حدوث تغيير
- Subscribe و Newsletter مثال: كلاسات تمثل كل منها مراقبًا يستجيب للإشعارات بطريقته الخاصة

Observer

- Database و File و Memory مثال: ثلاث طرق مختلفة لتخزين البيانات، يمكن استبدال أي منها بالأخرى دون تعديل الكود الذي يستخدمها
- Object هو نمط تصميم يُمثّل كائنًا صغيرًا لا يحتوي إلا على بيانات، ويُقارن بالقيمة لا بالهوية
- Money مثال: كلاس يمثل مبلغًا من المال مع عملته، ولا يمكن تغيير قيمته بعد إنشائه

Repository

- Value Object هو نمط تصميم يُستخدم لتمثيل مفاهيم بسيطة تُعرّف بقيمتها لا بهويتها
- Immutable يعني أن الكائن لا يمكن تغيير حالته بعد إنشائه، بل يجب إنشاء نسخة جديدة بالقيمة المطلوبة

Single Responsibility

- Class هو مبدأ يقول إن كل كلاس يجب أن يكون مسؤولًا عن شيء واحد فقط، وأن يكون لديه سبب واحد فقط للتغيير
- User و Repository و Validator مثال: كلاسات منفصلة، كل منها مسؤول عن جزء محدد من المنطق
- Open / Closed هو مبدأ يقول إن الكلاسات يجب أن تكون مفتوحة للتوسع، ومغلقة للتعديل
- Interface هو الأداة الأساسية التي نستخدمها لتطبيق هذا المبدأ، بدلًا من تعديل الكلاس الأصلي
- Programs to Interface هو مبدأ يقول إن الكود يجب أن يعتمد على Interfaces بدلًا من Classes
- Class هو الأداة التي نستخدمها لتطبيق هذا المبدأ، بدلًا من تعديل الكلاس الأصلي
- Favor Composition هو مبدأ يقول إن Composition أفضل من Inheritance في أغلب الحالات
- Inheritance هو الأداة التي نستخدمها لتطبيق هذا المبدأ، بدلًا من Composition
- "is-a" و "has-a" مثال: "Car has-a Engine" أفضل من "Car is-a Engine"

الأخطاء الشائعة وكيفية تجنبها

- "this->" عند استخدام Property داخل Class، يجب أن نكتب "$this->" قبل اسمها، وإلا فلن يعمل الكود كما نتوقع
- "new" عند استخدام Interface أو Abstract Class، لا يمكن كتابة "new" معهما، لأنهما لا يمكن إنشاء Object منهما مباشرة
- Private عند محاولة الوصول إلى Property أو Method من نوع Private من خارج Class، سيظهر خطأ Fatal Error
- "parent::__construct()" عند وراثة Class، يجب أن نتذكر استدعاء Constructor الأب يدويًا، وإلا فلن يتم تهيئة الخصائص الموروثة
- "readonly" عند محاولة تعديل Property من نوع readonly بعد تهيئتها، سيظهر خطأ Error من النوع Error

مقدمة الترجمة

- Object و Class الفرق بين
- Constructor و Objects الغرض من
- "this$" الكلمة المفتاحية
- Type Hinting و Polymorphism المفهوم
- Inheritance و Composition الفرق بين
- Dependency Injection و Traits الهدف
- PHP 8 و Magic Methods الميزات
- Design Patterns الأنماط